Painel do Negócio — Manual técnico e funcional

Documento de passagem do projeto para manutenção, evolução ou reconstrução.

Atualizado: 13 de outubro de 2026, referência de calendário Africa/Maputo. **Código de referência:** commit 47148d4 da branch publicada `main`. **Repositório:** https://github.com/BRAVOS23/Painel-Shopify- **Aplicação publicada:** https://painel-shopify.lobojose348.workers.dev/

Este documento descreve a implementação existente, as decisões confirmadas pelo proprietário e as limitações conhecidas. Não contém tokens, palavras-passe, chaves privadas, backups de clientes ou valores de Secrets. Configuração instalada numa plataforma não deve ser presumida apenas porque aparece num exemplo do repositório: é necessário verificá-la com o proprietário e na consola correspondente.

Referência principal: este manual substitui afirmações de estado antigas de README.md, SETUP.md e CLOUDFLARE.md, quando houver divergência. docs/UI-UX.md conserva também o histórico das intervenções; entradas antigas desse histórico não representam necessariamente a interface atual.

Índice

1. Objetivo e decisões de produto
2. Funcionalidades e experiência de utilização
3. Regras de pedidos, vendas e stock
4. Cálculos financeiros e gráficos
5. Arquitetura e mapa dos ficheiros
6. Modelo de dados e concorrência
7. API e contratos
8. Integração Shopify
9. Integração Meta Ads
10. Segurança, autenticação e privacidade
11. Configuração e ambientes
12. Desenvolvimento, compilação e publicação
13. Testes e critérios de aceitação
14. Operação e diagnóstico
15. Backups, migrações e recuperação
16. Limitações e trabalho pendente
17. Como reconstruir do zero
18. Lista de passagem ao próximo programador

1. Objetivo e decisões de produto

O painel gere um negócio de comércio eletrónico com pagamento na entrega, também chamado COD. Centraliza pedidos e organização de entregas, catálogo e stock local, custos, gastos com publicidade e estimativas de rentabilidade. A interface está em português; a moeda do negócio é MZN, apresentada como MT, e o fuso definido para o negócio é Africa/Maputo.

A Shopify é a origem dos pedidos e dos nomes de produtos vinculados. A Meta fornece gastos com anúncios. O painel acrescenta organização local, reagendamentos, custos dos produtos, entregadores e stock por local.

Decisões confirmadas que devem ser preservadas:

- Orçamento atual zero: Cloudflare Workers Free e Firebase Spark. Não ativar planos pagos automaticamente para resolver falhas.
- Navegação principal: Resumo, Pedidos e Resultados. Conta, integrações, aparência e dados ficam no menu de utilizador ⋯.
- A página de fluxo de caixa deixou de ser a entrada do produto. Os movimentos antigos continuam armazenados e incluídos nos backups.
- Reagendamentos podem atravessar meses sem perder o pedido nem o percurso anterior.
- Nomes de produtos podem acompanhar a Shopify; custo, fotografia e stock local pertencem ao painel. O inventário Shopify não deve ser sincronizado para o stock local.
- Uma compra/conversão reportada pela Meta não é uma venda do painel.
- Concluir um pedido ligado à Shopify deve solicitar processamento e registo de pagamento recebido, e aguardar confirmação dos dois estados.
- Os diálogos nativos recorrentes de confirmação de Concluir/Cancelar Shopify foram retirados. Não reintroduzir estes avisos por rotina. Há confirmações em outras ações, como apagar, importar e desligar.
- A limpeza em massa de registos manuais foi retirada. A restauração de backups produzidos por essa antiga funcionalidade permanece disponível.
- Histórico de gastos é uma área de consulta. O botão redundante de registar gasto dentro dessa área foi retirado; o registo fica na sua própria aba.

O painel é um piloto publicado e utilizado pelo proprietário. Não é uma plataforma comercial completa com assinaturas, faturação, equipas e requisitos de distribuição integralmente resolvidos.

2. Funcionalidades e experiência de utilização

2.1 Acesso e conta

Login por email/palavra-passe, criação de conta, recuperação de acesso e verificação de email. Em produção, a API exige email verificado. Cada UID Firebase representa o seu próprio negócio. Não há membros convidados, equipas nem papéis de colaborador.

Uma sessão reconhecida cuja base de dados esteja indisponível apresenta uma página de recuperação com Tentar novamente e Entrar noutra conta. Não deve exibir um erro de integração no formulário antes de o utilizador iniciar sessão.

O menu ⋯ contém conta, tema claro/escuro, opção de instalar a PWA quando disponível, integrações, backups e migração dos dados antigos. Sair é impedido enquanto existem alterações por guardar.

2.2 Resumo

Mostra pedidos em aberto, entregas concluídas, receita e lucro estimado para o mês escolhido, próximas entregas e atalhos para registar pedidos, organizar remarcados, consultar produtos e gerir integrações. Alerta para remarcados em meses anteriores e produtos sem custo definido.

2.3 Pedidos

A aba Pedidos tem registo de pedido, seleção de mês e grupos Semana 1 a Semana 4, Remarketing e Províncias. Há pesquisa por contacto, local e produto, filtro de estado, contacto telefónico/WhatsApp, ações de concluir, cancelar, editar, apagar, reagendar, mover de grupo e PDF.

Campos essenciais aparecem primeiro; origem, descrição, entregador, comissão e estado ficam nos detalhes. No móvel, cartões recolhem ações secundárias. A seleção para PDF é destinada aos pedidos da semana/grupo em consulta e não deve continuar inadvertidamente ao mudar de semana.

Relatórios mostram indicadores, receita concluída, evolução dos pedidos, distribuição por estado, origem, unidades vendidas, entregas por pessoa e a lista do período. **A lista e os indicadores seguem o mesmo intervalo**, não a semana ativa do formulário. O intervalo inicial é este mês. Listas admitem pesquisa e estado; o CSV corresponde à lista filtrada.

Meses anteriores abrem por padrão o mês completo. Têm seletores de mês, semana/grupo e estado, pesquisa, limpeza de filtros, resumo, gráfico diário da receita, CSV e reagendamento. Pedidos transferidos para outro mês continuam visíveis no histórico, mas não geram receita no mês de origem. A indicação Transferido mostra a data atual da entrega.

No telemóvel, as listas de relatórios e histórico mostram primeiro informação essencial; Ver detalhes revela fonte, local, quantidade e entregador.

2.4 Resultados

Subabas: Resultados, Registar gastos, Produtos e Histórico.

Resultados tem intervalo, filtro por produto cadastrado, receita, lucro estimado, ROI, CPA, ROAS, aviso de custos em falta, explicação recolhível, custos discriminados e gráficos. Receita e gasto total usam linhas com comparação ao tocar/hover e modo Acumulado. Alternar Acumulado redesenha o gráfico local; não deve depender de nova consulta ao servidor. A distribuição de custos usa gráfico circular.

Registar gastos apresenta uma categoria de cada vez: Anúncios, Outros ou Loja. Anúncios e Outros podem cobrir um intervalo de dias; Loja admite gasto único ou mensal recorrente.

Produtos apresenta catálogo pesquisável, criação/edição em diálogo, fotografia comprimida, custo unitário, stock por local e vínculo Shopify. Associar nomes permite ligar nomes históricos a um produto do catálogo sem reescrever o histórico financeiro.

Histórico reúne gastos manuais e Meta, com pesquisa, categoria, valores e total do filtro. Gastos Meta identificam a campanha e são geridos pela sincronização: não apresentam edição/apagamento manual de valor, que seria substituído na próxima importação. Gastos manuais continuam editáveis e removíveis individualmente.

2.5 Mobile e instalação

Fonte Inter distribuída localmente, valores com números tabulares, temas claro/escuro e alvos de toque geralmente de pelo menos 44 px. Até 1023 px, a navegação principal fica no fundo e as subabas ficam próximas do topo. Os calendários móveis abrem centrados e limitados ao ecrã. Diálogos têm nomes acessíveis, fecho com Escape e tratamento de foco.

A instalação é uma PWA com manifesto, ícones e service worker. Não existe APK nativo ou publicação nas lojas Android/iOS. Quando beforeinstallprompt não está disponível, a conta mostra instruções de instalação para o navegador/sistema.

Internet continua obrigatória para login, consulta e gravação dos dados do negócio. O service worker guarda a interface pública; não transforma o sistema numa aplicação de dados offline.

3. Regras de pedidos, vendas e stock

3.1 Estados e confirmação de venda

Estados armazenados: `pendente`, `remarcado`, `concluido`, `cancelado`.

A regra partilhada `web/sales.mjs:completedSale` aceita receita apenas quando o registo tem status concluido e satisfaz um dos seguintes critérios:

- É um registo manual sem shopifyOrderId; ou
- Foi explicitamente confirmado no painel, com panelConfirmed=true; ou
- Tem shopifyFinancialStatus=paid e shopifyFulfillmentStatus=fulfilled.

Importação Shopify por API/CSV só conclui automaticamente quando pago e processado. Pago sem processado e processado sem pago permanecem pendentes. Cancelamento explícito, refunded completo ou voided são tratados como cancelado pela normalização atual. Esta última regra não substitui um sistema contabilístico de reembolsos; deve ser reavaliada se esse âmbito for implementado.

Registos antigos classificados como concluídos mas sem prova podem aparecer como A verificar e ficar fora da receita. Não alterar todos os estados em massa para melhorar um número apresentado: conferir a fonte e preservar a confirmação explícita válida.

Pedidos distintos são contados por shopifyOrderId; registos manuais são contados por id. Um pedido Shopify pode ter várias linhas de produto, mas conta uma vez no número de pedidos/CPA, origem e entregas por pessoa. Valores, unidades e comissões são somados por linha.

3.2 Concluir e Cancelar

Num pedido ligado à Shopify, Concluir:

1. Confirma que o pedido pertence ao UID autenticado e adquire lease Shopify.
2. Lê o estado real; recusa cancelados e pagamentos incompatíveis.
3. Se necessário, cria o processamento a partir dos fulfillment orders suportados.
4. Se necessário e permitido, usa orderMarkAsPaid para registar pagamento recebido.
5. Lê novamente e só confirma localmente quando processado e pago.

Não utiliza captura/cobrança de cartão. Deve ser usado após receber o pagamento. Se o processamento funcionar e o pagamento falhar, a ação pode ser repetida para retomar apenas o que falta. Um retorno HTTP 200 GraphQL, sozinho, não prova sucesso: os userErrors também são verificados.

Cancelar usa orderCancel com refund=false, restock=true e notifyCustomer=false. Não faz reembolso automático; pode repor inventário na Shopify. Isso não sincroniza inventário para o stock do painel.

Enquanto a plataforma não confirmar, a API devolve 202 e o painel mantém o estado anterior, indica atualização e consulta confirmação por GET. O GET nunca pede novo processamento/pagamento/cancelamento à Shopify. A API atualiza todos os registos locais existentes

desse order ID, incluindo IDs vindos de CSV/backup, sem criar novas linhas nessa ação.

A edição do formulário de um pedido Shopify não permite mudar o seu estado remoto indiretamente; o campo fica bloqueado e indica Concluir/Cancelar. Sem uma ligação Shopify, registos com ID Shopify podem ser alterados só no painel após confirmação específica desse modo local.

3.3 Reagendamento e datas

Semana por data: dias 1–7 → semana1; 8–14 → semana2; 15–21 → semana3; restantes → semana4. Grupos também podem ser escolhidos manualmente.

Um reagendamento conserva id e valor; guarda dataPedido se ainda não existir; altera data, mes, horario e grupo; define remarcado; acrescenta uma entrada de percurso. As últimas 20 entradas são preservadas. Só pendentes/remarcados podem ser reagendados.

A sincronização preserva campos operacionais locais: horário, entregador, comissão, grupo, mês, descrição, data de entrega, data original e histórico. A data financeira usada nos filtros é **a data atual registada na linha do pedido**, que pode ser a entrega reagendada. Não há um campo contabilístico independente de data de recebimento/conclusão.

3.4 Catálogo e stock

Cada produto local tem id próprio, nome, custo, fotografia, aliases e locais de stock. O vínculo Shopify acrescenta shopifyProductId e shopifyShop, sem substituir custo/foto/quantidades. Novos produtos criados pela ligação começam com stock vazio e custo por definir.

Produtos vinculados têm nome gerido na Shopify; títulos antigos tornam-se aliases. Não são consultados nem escritos campos de inventário Shopify para este vínculo.

O stock local é o total das quantidades introduzidas por local. Não existe um livro de movimentos de stock nem dedução automática garantida em Concluir. Não apresentar stock local como saldo reconciliado da Shopify.

4. Cálculos financeiros e gráficos

A referência financeira é server/model.mjs:calculateMetrics. O frontend calcula séries diárias para desenhar gráficos, que devem permanecer equivalentes ao backend.

Indicador	Regra atual
Receita	Soma de valor das linhas de vendas confirmadas, dentro do período e produto filtrado.
Custo de mercadoria	Custo unitário atual do produto resolvido × quantidade das linhas confirmadas.
Entregas	Soma de comissao das linhas confirmadas.
Anúncios	Gastos tipo ads atribuídos ao período; campanhas Meta incluídas mesmo sem produto no total geral.
Outros	Gastos tipo outros atribuídos ao período.
Loja	Gastos operacionais únicos na data; recorrentes distribuídos pelos dias do mês. Só no total geral.
Total de custos	Mercadoria + entregas + anúncios + outros + loja.

Indicador	Regra atual
Lucro estimado	Receita − total de custos.
CPA	Anúncios ÷ pedidos confirmados distintos; null se não houver vendas.
ROAS	Receita ÷ anúncios; null se não houver gasto de anúncios.
ROI	Lucro estimado ÷ total de custos × 100; null se não houver custos.

Exemplo de referência: receita 200 MT, quantidade 2, custo unitário 20 MT, entrega 10 MT, anúncios 40 MT, outros 20 MT e loja 10 MT → custos 120 MT, lucro 80 MT, CPA 40 MT, ROAS 5x e ROI aproximadamente 66,67%.

Um gasto de 70 MT de 10 a 16 de um mês conta 10 MT por dia. O cálculo usa dias inclusivos e sobreposição do intervalo. Um gasto mensal recorrente de 310 MT num mês de 31 dias conta 10 MT por dia a partir da sua data inicial. A recorrência não tem data de fim na implementação atual.

Ao filtrar um produto, os custos gerais da loja ficam fora; gastos gerais sem produto também não são distribuídos arbitrariamente por produtos. O aviso de custo em falta torna o lucro uma estimativa incompleta, não uma validação de margem.

CPA e ROAS usam todas as vendas confirmadas do período. Não representam atribuição de vendas a campanhas Meta individuais. Os gráficos Acumulado apenas somam a série já calculada; não mudam estes critérios.

Limites financeiros relevantes:

- Os custos unitários vêm do catálogo atual, não de snapshots históricos por venda. Alterar um custo pode alterar estimativas históricas.
- A receita Shopify é preço dos itens × quantidade menos descontos dos itens. Não soma automaticamente frete/impostos nem reconcilia reembolsos parciais.
- Valores de entrega são por linha. Não repetir a comissão inteira em todas as linhas de um mesmo pedido sem uma decisão explícita de alocação.
- IDs determinísticos evitam repetir a mesma importação; não constituem uma deduplicação universal de um CSV antigo e linhas API com IDs diferentes para o mesmo pedido. Auditar coexistência antes de certificar totais.

Relatórios usam o período selecionado; séries longas agrupam por semana acima de 62 dias e por mês acima de 180. Meses anteriores usam receita diária apenas de linhas cuja data/mês atual pertence ao mês consultado. As taxas de sucesso comparam concluídos com concluídos + cancelados; pendentes não entram nesse denominador.

5. Arquitetura e mapa dos ficheiros

Fluxo principal: navegador → API autenticada Cloudflare Worker → Firestore por HTTPS. O navegador usa Firebase Auth para obter o ID token; esse token acompanha a API em Authorization: Bearer. Tokens Shopify/Meta ficam cifrados no servidor e não são enviados ao frontend.

O Worker também serve os assets da interface e executa Cron. Um backend Express existe para desenvolvimento e testes; parte das rotas e regras é partilhada. Não assumir equivalência completa dos schedulers Node e Worker.

Ficheiro/área	Responsabilidade
painel-negocio-3.html	Estrutura da interface e script clássico principal de pedidos, formulários, calendários, CSV, PDF e gráficos.
web/app.mjs	Firebase Auth, API, estado/baseline, gravações, sessões, integrações e verificação de contagens.
web/experience.mjs	Menu de conta, Resumo, diálogos e reagendamentos entre meses.
web/usability.mjs	Simplificação dos formulários, catálogo em diálogo, acessibilidade e composição da interface.
web/experience.css	Tipografia, temas, layout responsivo, cartões e diálogos.
web/orders.mjs	Validação de datas e transformação de reagendamento.
web/sales.mjs	Critério de venda confirmada e contagem distinta de pedidos.
web/order-reports.mjs	Resumos e séries de pedidos, gráficos de relatórios/histórico.
web/catalog.mjs	Resolução de nomes/aliases e campanhas Meta com datas finais.
web/reconcile.mjs	Diálogo de associação de nomes históricos.
web/shopify-catalog.mjs	Interface para selecionar vínculos Shopify/produto local.
web/record-schema.mjs	Validação partilhada dos registos e backups.
web/manual-cleanup.mjs	Compatibilidade para restaurar backups da antiga limpeza; não inclui o antigo botão de apagar tudo.
web/install.mjs	Prompt de instalação PWA e instruções alternativas.
web/sw.template.js	Service worker, cache só de conteúdo público.
server/api.mjs	Rotas partilhadas, autenticação, OAuth, webhooks, métricas e ações dos pedidos.
server/store.mjs	Negócios, persistência, revisões, importação idempotente, leases e cache opcional.
server/model.mjs	Normalização de pedidos/gastos e cálculo financeiro.
server/providers.mjs	GraphQL Shopify, API Meta, paginação, normalização e erros sanitizados.
server/security.mjs	AES-GCM, assinaturas HMAC, domínios, IDs e projeção pública das ligações.
server/catalog.mjs	Vínculo e atualização de nomes Shopify preservando stock/custos locais.
server/status-audit.mjs	Verificação dos estados atuais de pedidos já guardados, incluindo históricos.
server/index.mjs, app.mjs, config.mjs, sync.mjs	Backend Node local, configuração e scheduler alternativo.
worker/index.mjs	Entradas fetch/scheduled, runtime, bindings e orçamento de chamadas.

Ficheiro/área	Responsabilidade
worker/router.mjs	Adaptação das rotas/middleware para Request/Response.
worker/firestore.mjs	Cliente REST Firestore, tipos, consultas e transações.
worker/google.mjs	Identidade de serviço Google e verificação oficial dos utilizadores Firebase.
worker/sync.mjs	Jobs em lotes, cursores, webhooks e alternância entre negócios/provedores.
scripts/build.mjs	Gera dist, bundle, dashboard clássico, CSS, versão e service worker.
scripts/dev.mjs, export-emulators.mjs	Inicialização e snapshots locais dos emuladores.
scripts/wrangler.mjs	Wrapper CLI com configuração/logs locais.
wrangler.jsonc	Worker, assets, cron, variáveis públicas e ambiente local.
firebase.json, firestore.rules	Emuladores e negação de acesso direto ao banco.
public/	Manifesto, ícones e fontes locais.
legacy/	Protótipo anterior, conservado para migração. Não é a aplicação atual.
tests/	Testes unitários, integração, Workerd e criptografia.
dist/	Resultado gerado e ignorado pelo Git; não editar como fonte.

A interface é híbrida: módulos ESM e funções globais do script clássico. web/app.mjs carrega dashboard.js após autenticar/adotar o estado e depois inicializa a experiência. Ao mover elementos DOM, evitar procurá-los por document.getElementById enquanto estão num fragmento desligado do documento.

6. Modelo de dados e concorrência

6.1 Organização Firestore

```
businesses/{uid}
  fluxo/{recordId}
  vendas/{recordId}
  produtos/{recordId}
  ads/{recordId}
  operacional/{recordId}
  connections/shopify
  connections/meta
  leases/{provider}
oauthStates/{randomState}
shopLinks/{canonicalShopDomain}
webhookQueue/{deduplicatedWebhookHash}
system/freeScheduler
```

O documento businesses/{uid} contém ownerId, email, currency=MZN, timezone=Africa/Maputo, createdAt e dataVersion. O ownerId deve corresponder ao UID autenticado. A dataVersion é incrementada em transações de alteração de dados e invalida o estado/cache.

Cada documento das cinco coleções de registos é um envelope:

```
{
```

```
  "record": { "id": "manual_exemplo", "data": "2026-10-13", "...": "campos da coleção" },
  "revision": 1,
  "updatedAt": "2026-10-13T08:00:00.000Z",
  "sourceUpdatedAt": "campo opcional para importações"
}
```

A API projeta o conteúdo de record e acrescenta _revision. Esse campo não pertence ao registo editável; é separado ao guardar. O formato completo autorizado é definido em web/record-schema.mjs, não nos exemplos resumidos deste documento.

6.2 Campos de negócio

Coleção	Campos principais
fluxo	id, data, tipo entrada/saida, valor, descricao, observacao. Preservação do legado.
vendas	id, data, mes, semana, horario, fonte, contacto, local, descricao, produto, quantidade, valor, delivery, comissao, status.
vendas — origem/prova	shopifyOrderId, shopifyLineItemId, sourceCurrency, panelConfirmed, shopifyFinancialStatus, shopifyFulfillmentStatus.
vendas — percurso	dataPedido e reagendamentos[{de,para,semanaAnterior,semanaNova,horario,alteradoEm}].
produtos	id, nome, custo, foto, locais[{id,local,quantidade}], aliases, shopifyProductId, shopifyShop, costConfigured.
ads	id, data, dataFim opcional, tipo ads/outros, nome opcional, produto e valor.
ads — Meta	source=meta, campaignId, campaignName, accountId, sourceCurrency, sourceValue, conversionRate.
operacional	id, nome, valor, data, recorrente booleano.

IDs de linhas API Shopify: sh_{orderId}_{lineItemId}. Produtos novos vinculados: sp_{shopifyProductId}. Gastos Meta: meta_{accountId}_{campaignId}_{date_start}. Não mudar estes formatos sem um plano de migração/idempotência.

Limites de validação relevantes: IDs de até 180 caracteres alfanuméricos/underscore/hífen; texto comum até 4.000 caracteres; registo serializado até aproximadamente 600.000 caracteres; fotografia base64 até 550.000 caracteres; até 50 aliases e 50 locais; até 20 entradas de reagendamento. Números monetários devem ser finitos e não negativos; quantidades por local são inteiros não negativos. O schema atual não exige de forma completa todos os campos de todas as coleções: reforçar obrigatoriedade numa evolução exige compatibilidade/migração de registos antigos.

6.3 Ligações e jobs

Shopify guarda credentials cifradas, shop canónica, connectedAt, lastSync, lastError, progress, salesPolicyVersion, catálogo e statusAudit. Meta guarda credentials cifradas, connectedAt, expiresAt, conta, moeda, câmbio, fuso, lastSync, progress, erros e campanhas não associadas.

statusAudit guarda IDs privados do job, cursor, offset de linhas, prova temporal, contagens, início/fim e permissões históricas. A resposta pública mostra apenas progresso e necessidade de read_all_orders; nunca lista os IDs privados do job ou credentials.

Um lease tem token aleatório e until. A duração atual é 10 minutos. A libertação só apaga o lease se o token ainda corresponder ao dono da execução.

6.4 Gravação e conflitos

O navegador mantém baseline, desired, dirty, busy e failure. Guarda alterações em lotes de até cinco registos, procurando manter o corpo abaixo de cerca de 700 KB. A API aceita até 100 mudanças por chamada, mas o frontend usa lotes menores.

Cada mudança leva id, revision esperada e record completo ou null para apagar. Uma transação lê revisões, recusa alterações obsoletas com 409, aplica mudanças e incrementa dataVersion. Não remover a revisão para contornar um conflito.

Importações ignoram versões de origem mais antigas e evitam regravar registos iguais com o mesmo timestamp. Preservam os campos operacionais definidos. Um cancelamento de origem prevalece sobre a confirmação explícita; uma fonte pendente não deve apagar uma confirmação local válida.

O estado é consultado aproximadamente a cada 60 segundos com a aba visível e ao voltar à aba. ETag tem UID + dataVersion. Uma resposta 304 evita reler coleções. Ligações são consultadas aproximadamente a cada cinco minutos.

No Worker, estado e métricas partilham cache em memória por projeto/ambiente e UID/dataVersion: TTL cinco minutos, até oito MB/20 entradas e limite aproximado de dois MB por entrada. Workers frios voltam ao Firestore. Este cache não é um banco de dados nem substitui as revisões.

7. API e contratos

Todas as rotas /api de dados passam pelo middleware de autenticação, exceto health e config. O proprietário vem do ID token verificado. Não há parâmetro ownerId aceite do cliente para escolher outro negócio.

Método e rota	Função
GET /api/health	Saúde básica e modo emulator/production; não prova todas as integrações.
GET /api/config	Configuração Firebase Web pública e indicação de emuladores.
GET /api/state	Cinco coleções, records com _revision; ETag e 304.
POST /api/data/:collection	Gravar/apagar por changes e revisão.
GET /api/metrics?start=...&end=...&product=...	Métricas; datas válidas e intervalo limitado a 366 dias.
GET /api/connections	Metadados públicos e disponibilidade de configuração.
DELETE /api/connections/:provider	Desligar, preservando registos já importados.
POST /api/sync/:provider	Executar/retomar sincronização; provider shopify ou meta.
POST /api/shopify/orders/:id/status	Solicitar concluido/cancelado e verificar confirmação.
GET /api/shopify/orders/:id/status?status=...	Consultar confirmação sem nova mutação Shopify.

Método e rota	Função
POST /api/shopify/orders/reconcile	Consultar estados dos pedidos guardados; restart=true inicia nova passagem.
GET /api/shopify/products?after=...	Lista paginada de nomes/IDs para vínculo.
POST /api/shopify/products/link	Vincular produtos com IDs/revisões.
GET /api/meta/accounts	Contas disponíveis para o token da ligação.
POST /api/meta/account	Selecionar conta e conversionRate, iniciar sync.
POST /api/oauth/:provider	Criar tentativa OAuth e devolver URL de autorização.
GET /oauth/:provider/callback	Validar tentativa/assinatura, trocar código e guardar token cifrado.
POST /webhooks/shopify	Verificar HMAC, domínio e deduplicar/guardar job.

Exemplo sintético de gravação:

```
{
  "changes": [
    {"id":"produto_teste","revision":0,"record":{"id":"produto_teste","nome":"Produto teste","custo":2
    0,"foto":"","locais":[]}}
  ]
}
```

A resposta tem revisions:[{id,revision}]. Novo registo usa revisão zero. Apagar usa record:null com a revisão atual.

Códigos relevantes: 400 validação, 401 sessão ausente/inválida, 403 email não verificado, 409 conflito/lease/estado incompatível, 422 limite/condição de dados não suportada, 429 excesso de pedidos/quota, 502 falha da plataforma, 503 configuração/serviço indisponível. Existe rate limit por UID de 200 chamadas por minuto, em memória do runtime; não é um rate limiter distribuído global.

Corpos JSON têm limite de um MB; webhook bruto tem limite de cinco MB. Os erros retornam mensagens sanitizadas e, para Firestore, códigos diagnósticos fixos quando reconhecidos.

8. Integração Shopify

8.1 Instalação e OAuth

Aplicação utilizada pelo proprietário: Ponte Painel. O nome comercial não substitui a identificação/configuração efetiva do app no painel Shopify Dev.

Callback de produção: https://painel-shopify.lobojose348.workers.dev/oauth/shopify/callback. Webhook: https://painel-shopify.lobojose348.workers.dev/webhooks/shopify. APP_BASE_URL deve ser o URL HTTPS publicado, sem barra final; a configuração da Shopify deve corresponder exatamente.

Permissões base: read_products, read_orders, write_orders, read_merchant_managed_fulfillment_orders e write_merchant_managed_fulfillment_orders. Aprovações de dados protegidos e casos de locais/prestadores de terceiros podem exigir tratamento adicional.

read_all_orders é solicitado de forma condicional para leitura de históricos, quando a verificação deteta necessidade ou a ligação já foi configurada para esse acesso. A Shopify também deve autorizar esse acesso no app; acrescentar o texto ao OAuth não concede uma aprovação inexistente.

O domínio é original/canónico .myshopify.com, não o domínio público da montra. Aliases diferentes só são aceites quando a identidade Shopify confirma que são a mesma loja. shopLinks impede vincular a mesma loja a dois negócios.

A tentativa OAuth tem state aleatório de 64 caracteres hexadecimais, dono, provider, domínio e expiração de dez minutos. É consumida uma vez em transação. Callback Shopify exige HMAC; não desativar a assinatura para resolver um erro de domínio.

Reautorizar a mesma loja preserva progresso de importação e catálogo. Renovar acesso histórico reinicia a verificação de estados. Uma loja diferente não deve herdar cursores de outra loja. O token é cifrado antes de ir para Firestore.

8.2 Importação e webhooks

A versão API configurada é GraphQL 2026-07. A importação inicial revê cerca de 59 dias. Depois usa alterações desde lastSync, com sobreposição de cinco minutos no Worker. A revisão da política de vendas usa salesPolicyVersion=2 e uma passagem de 59 dias. No Node, a sobreposição incremental continua a ser 24 horas.

Cada página Worker consulta um pedido e grava até cinco linhas por execução. Pedidos com mais linhas continuam pelo seu ID, offset e timestamp. Há proteção contra uma mudança no pedido entre lotes. Pedidos com mais de 100 itens são recusados; paginação adicional desse caso ainda não foi implementada.

Webhooks: orders/create, orders/updated, orders/cancelled e app/uninstalled. A criação de subscrições consulta existentes e reutiliza tópico + callback HTTPS + JSON, sem duplicar indiscriminadamente.

A assinatura usa o corpo bruto recebido. A fila é deduplicada pelo hash de shop + webhook ID. O payload não é a confirmação definitiva: o worker volta a ler o pedido atual na API. A desinstalação remove a ligação/shopLink, mantendo os registos já importados.

8.3 Verificação de contagens antigas

server/status-audit.mjs consulta estados de IDs **já guardados no painel**, sem limitar a leitura à janela incremental de 59 dias. Lê até cinco pedidos Shopify por chamada e atualiza até dez linhas locais, retomando offset quando necessário.

Só consulta estado financeiro, processamento, cancelamento e timestamps. Não cria processamento, não marca pagamento, não cancela na Shopify e não cria pedidos locais novos. Conserva dados operacionais e confirmações explícitas conforme a regra de reconciliação.

Há progresso no relatório e um botão Verificar contagem na Shopify. Ao abrir Relatórios, uma passagem pendente pode ser retomada automaticamente. O Cron também trata a primeira verificação de ligações antigas e jobs incompletos. O navegador atualiza o estado em blocos para reduzir leituras e para ao ficar oculto; o Cron pode continuar.

Pedidos ausentes/inacessíveis são contabilizados como indisponíveis, com os registos locais preservados. O job verifica accessScopes e distingue a falta de read_all_orders de outros casos de indisponibilidade. A reautorização para acesso histórico pede a permissão e permite repetir a passagem.

Importante: esta verificação não importa todos os pedidos em falta de um ano. Um filtro anual mostra apenas os registos que já foram carregados no painel e pertencem àquele intervalo. Não certificar um total anual sem comparar cobertura com a Shopify.

Último incidente reportado: o proprietário observou 131 pedidos, 97 cancelados e apenas oito concluídos num filtro anual, apesar de afirmar haver mais pedidos processados e pagos. Foi implementada e testada esta nova verificação, mas **o total real da conta não foi confirmado pelo**

programador. Não tratar os números do cenário simulado de testes como resultados da loja.

9. Integração Meta Ads

Aplicação utilizada pelo proprietário: Painel do Negócio, caso de uso Marketing API. Callback: https://painel-shopify.lobojose348.workers.dev/oauth/meta/callback. A configuração usa META_API_VERSION=v24.0 e scope ads_read. Verificar suporte/revisão da versão e tipo de login antes de distribuir a clientes externos.

O token fica cifrado no servidor. A reautorização preserva a conta selecionada, moeda, câmbio, lastSync e cursor, substituindo credenciais/expiração. Selecionar outra conta após uma conta já ativa é recusado sem desligar primeiro, para evitar misturar contas inadvertidamente.

A importação consulta Insights por campanha/dia, nos últimos 30 dias, com campaign_id, campaign_name, spend, date_start e date_stop. Não consulta compras para gerar vendas. A janela é revisitada porque a Meta pode corrigir gastos passados. O Worker usa páginas de até cinco resultados e cursor explícito; não segue URLs de paginação arbitrárias fora do domínio oficial.

Se a conta não usar MZN, a taxa é explícita e positiva: 1 unidade da moeda de origem = X MZN. A taxa fica guardada na ligação e em cada gasto. Não há câmbio diário automático. Guardar uma nova taxa pode recalcular despesas reimportadas da janela, logo pode alterar resultados anteriores. A interface repõe a taxa após reload e não a substitui enquanto o utilizador está a editar.

Associação de campanha: prioridade ao nome exato/normalizado do produto ou alias; admite uma data válida no fim, como DD/MM/AA, DD/MM/AAAA ou AAAA-MM-DD, com alguns separadores/parênteses. Não é pesquisa por substring livre. Nomes ambíguos não são associados arbitrariamente. Um conjunto anunciado deve existir como produto conjunto para ser associado por esse nome.

Campanhas sem associação continuam no gasto total geral e são indicadas. O nome original da campanha é preservado. Uma associação nova pode ser usada nos cálculos sem reescrever o gasto bruto.

A sincronização não elimina gastos manuais semelhantes. Evitar lançar manualmente o mesmo gasto importado. Desligar a integração também não apaga o histórico importado.

10. Segurança, autenticação e privacidade

- Firebase ID token é verificado pelo serviço oficial, incluindo projeto, identidade, validade e revogação. Descodificar um JWT não é verificá-lo.
- Email verificado é obrigatório em produção; o requisito é relaxado apenas nos emuladores.
- O UID autenticado determina o negócio. Consultas/mutações são limitadas aos registos desse negócio.
- firestore.rules nega leitura/escrita direta de todos os documentos pelo cliente. Administração é do backend, que aplica o isolamento.
- Credenciais de integração usam AES-256-GCM: iv, tag e data em base64. INTEGRATION_ENCRYPTION_KEY tem 32 bytes aleatórios, representados em base64. A cifragem deve usar a entrada UTF-8 correta também no runtime Workerd.
- HMAC verifica OAuth Shopify e webhooks. Credenciais, respostas completas das plataformas e URLs OAuth com códigos não devem ir para logs públicos.
- O token Meta usa appsecret_proof nas chamadas relevantes do servidor.

- IDs e domínios são restritos; campos desconhecidos são rejeitados. Datas impossíveis são recusadas. Fotografias são data URLs JPEG/PNG/WebP em base64, não URLs executáveis.
- Headers incluem no-sniff, no-referrer, anti-frame e CSP. A CSP ainda admite unsafe-inline por causa da interface clássica; melhorar isto exige refatoração dos handlers/scripts.
- Service worker não guarda /api, /oauth, /webhooks, tokens ou respostas privadas. A persistência de sessão Firebase é responsabilidade do SDK de autenticação, não uma promessa de cache dos dados do negócio.
- Alterações ainda não guardadas ficam na memória da aba. Fechar uma aba ou perder o processo pode perdê-las; beforeunload apenas avisa.

A chave de cifragem não pode ser trocada sem plano: tokens existentes deixam de ser legíveis. Rotação exige recifrar com a chave antiga ou reautorizar ligações de forma controlada. O JSON da conta de serviço Firebase é um segredo administrativo, não a configuração Firebase Web pública.

Ainda faltam política/fluxo completo de retenção e eliminação de conta, requisitos de conformidade das plataformas para distribuição, auditoria de negócio e mecanismos de equipas/perfis. Não apresentar a aplicação como já certificada para esses requisitos.

11. Configuração e ambientes

11.1 Identificadores e variáveis públicas

Nome	Produção atual/referência
Worker	painel-shopify
FIREBASE_PROJECT_ID	painel-cash-on-delivery
FIREBASE_AUTH_DOMAIN	painel-cash-on-delivery.firebaseapp.com
FIREBASE_WEB_API_KEY	Configuração pública em wrangler.jsonc e config/firebase.production.json; não repetir aqui.
APP_BASE_URL	https://painel-shopify.lobojose348.workers.dev
USE_FIREBASE_EMULATORS	false em produção; true apenas localmente.
SHOPIFY_API_VERSION	2026-07
META_API_VERSION	v24.0
compatibility_date	2026-10-12
compatibility_flags	nodejs_compat

11.2 Secrets

Nome	Conteúdo/função
FIREBASE_SERVICE_ACCOUNT	JSON completo da identidade Google do projeto correto, só no Worker.
INTEGRATION_ENCRYPTION_KEY	Chave de 32 bytes em base64, necessária para ler tokens cifrados existentes.
SHOPIFY_CLIENT_ID	ID do app; pode ser público, mas deve permanecer disponível na configuração do deploy.
SHOPIFY_CLIENT_SECRET	Segredo do app Shopify para OAuth/HMAC.
META_APP_ID	ID do app Meta, disponível na configuração.
META_APP_SECRET	Segredo do app Meta.

IDs podem ser variáveis normais, mas foram também tratados como Secrets no processo de configuração do proprietário. Não apagar uma binding assumindo que um ID está em wrangler.jsonc. O código não inclui valores secretos. Não usar dumps de ambiente para verificar configuração; verificar nomes/presença e testar operação segura.

No Node, a identidade Google usa Firebase Admin/Application Default Credentials; num cenário real, verificar credenciais e permissões IAM sem colocar ficheiros privados no Git. .env.example é exemplo local; config/production.env.example é referência de variáveis de produção. Estes ficheiros não comprovam que os Secrets foram instalados.

Para gerar uma chave nova num ambiente confiável:

```
node -e "console.log(require('crypto').randomBytes(32).toString('base64'))"
```

O resultado é secreto. Guardar no gestor de Secrets; não colar em chat, código fonte ou documentação. Não gerar uma chave nova para substituir a existente de uma instalação com tokens cifrados sem plano de recuperação.

12. Desenvolvimento, compilação e publicação

12.1 Arranque local

Requisitos: Node >=22, npm compatível com o lockfile e Java 21 para Firestore Emulator. Use o checkout existente; não é preciso um worktree extra.

```
npm ci
npm run dev
```

npm run dev compila a interface, inicia Auth/Firestore emulados no projeto demo-painel, aguarda disponibilidade e inicia Node na porta 3000. Auth usa 9099; Firestore usa 8080. Não ligar emuladores a um projeto real. Criar contas e registos sintéticos para desenvolvimento.

O encerramento normal exporta snapshots para .emulator-data; ficheiros auxiliares/snapshots anteriores ficam em .local. O encerramento abrupto pode perder alterações locais desde a última exportação. Não apagar esses diretórios para resolver um problema de interface.

Com os emuladores já ativos, num segundo terminal:

```
npm run dev:worker
```

Worker local usa 8787 e o ambiente local do Wrangler. Alternativa: iniciar npm run emulators separadamente e depois dev:worker, sem arrancar uma segunda cópia nas mesmas portas.

Não abrir o HTML por file:// nem servir só os ficheiros estáticos: autenticação e persistência dependem do backend.

12.2 Build

```
npm run build
```

scripts/build.mjs extrai o último script clássico do HTML para dist/assets/dashboard.js, gera bundle ESM de web/app.mjs com esbuild, copia CSS/fonts/ícones, gera HTML e service worker. Chart.js e jsPDF estão no bundle, sem dependência de CDN em runtime.

A versão painel-build é um hash de 12 caracteres do HTML, app e ficheiros listados em extra no build. **Ao acrescentar um módulo ou alterar regras relevantes para a versão publicada, conferir a lista usada no hash.** Preservar a posição do CSS usada pelo script; não editar dist como fonte.

Dependências fixadas no package.json/lockfile: Firebase Web 13.0.0, firebase-admin 14.5.0, Express 5.3.0, Chart.js 4.4.4, jsPDF 4.2.1, esbuild 0.28.2, firebase-tools 15.33.0 e Wrangler 4.149.0. Atualizações exigem testes, sobretudo compatibilidade Worker, cifragem e APIs das plataformas.

12.3 Publicação

Cloudflare usa o repositório GitHub e branch main. Configuração de referência: raiz do repositório, build npm run build e deploy npx wrangler deploy --env="". Não publicar como Pages estático: as rotas de servidor são essenciais.

```
npm run check:worker
```

É dry-run, não deploy. Publicação direta, quando o operador estiver autenticado e autorizado:

```
npm run deploy:worker
```

No fluxo utilizado, alterações testadas são commitadas e enviadas para main; a Cloudflare publica automaticamente. O checkout de desenvolvimento esteve na branch cloudflare-free, com push HEAD:main. Esse nome local não muda a branch que a Cloudflare acompanha.

Verificar que o commit chegou à branch certa e comparar meta painel-build do site com dist/index.html. Esse teste confirma a versão de assets publicada; não prova o estado das credenciais, os totais dos clientes ou o funcionamento de cada ação real.

Confirmar que APP_BASE_URL, callbacks, domínio autorizado Firebase e Secrets correspondem ao novo endereço. Não remover Secrets existentes ao atualizar variáveis. Não ativar Workers Paid/Blaze automaticamente.

13. Testes e critérios de aceitação

```
npm test
npm run test:integration
npm run test:worker
npm run check:worker
npm run test:worker:crypto
```

Integração requer emuladores ativos. Teste Worker requer também runtime local em 8787. O teste de cifragem usa fixture própria do Workerd; consultar tests/worker-crypto.mjs. Não executar esses testes apontando para o domínio de produção.

Suite	Prova principal
Unitários	Assinaturas, cifragem, normalização, moedas, aliases, métricas, datas e séries.
Integração Node	Auth/Firestore emulados, isolamento, revisões, OAuth, ações e auditoria de estados com provedores simulados.
Worker integration	API em Workerd e Firestore REST, tipos/transações, limites, cursor, jobs e auditoria.
Worker crypto	Compatibilidade real da cifragem com nodejs_compat/Workerd.
Dry-run	Build/bindings/empacotamento; não publica nem certifica CPU/quota de produção.

Última verificação funcional antes deste documento: 46 testes unitários, 25 verificações de integração e 15 verificações Worker passaram; build/dry-run passaram. Estes números são uma fotografia e mudam com novas suites. O teste crypto foi executado em intervenções anteriores, não novamente como parte desta produção de documentação.

Testes Chromium adicionais, executados em scripts temporários de desenvolvimento, cobriram quatro larguras 360/390/800/1280, claro/escuro, filtros, relatórios, stock por local, PDF, gráficos, sessão, instalação simulada, troca do acumulado com endpoint indisponível e verificação de contagem. **Esses scripts temporários não são uma suite de navegador versionada completa.** Não prometer que o futuro checkout os contém.

Critérios mínimos para regressão/reconstrução:

- Dois utilizadores não conseguem ler/alterar dados um do outro.
- Login/restauro/falha de sessão têm mensagens corretas e recuperação sem confundir integração com login.
- Alterar um registo com revisão antiga falha sem apagar a alteração mais recente.
- Reimportar não duplica a mesma linha nem gasto Meta.
- Reagendar de setembro para outubro preserva id, data original e histórico depois da sincronização.
- Processado sem pago e pago sem processado não entram automaticamente na receita.
- Concluir aguarda os dois estados; GET de confirmação não repete mutações.
- A verificação de contagem lê estados antigos, preserva dados operacionais e apresenta indisponíveis.
- A Meta com centenas de compras reportadas não cria vendas no painel.
- Campanha Produto + data corresponde ao produto; nomes ambíguos não são associados.
- Custo/foto/stock local sobrevivem ao vínculo e renomeação Shopify.
- Câmbio é preservado depois de guardar, recarregar e reautorizar a Meta.
- Gráfico e tabela respondem ao mesmo filtro; alternar acumulado regressa à série diária.
- Backup inválido é recusado antes da primeira gravação.
- Layout móvel não tem scroll horizontal; detalhes podem ser abertos por toque/teclado.

Operações com efeitos reais devem ser validadas apenas com contas/pedidos de teste e autorização adequada. As suites automatizadas existentes não alteram lojas ou contas de anúncios reais.

14. Operação e diagnóstico

14.1 Agendamento e quotas

Cron de produção: uma execução por minuto. Prioridade para um webhook pendente; sem webhook, alterna Shopify/Meta de um negócio e avança para o seguinte. Uma verificação de estados pendente pode anteceder catálogo/importação Shopify. Catálogo vinculado é reconciliado diariamente em lotes; falhas recebem estado/backoff para não bloquear pedidos indefinidamente. Uma janela de sincronização concluída tem intervalo mínimo de cerca de 15 minutos.

Isto não significa que todos os pedidos de todos os negócios sincronizam a cada minuto. Vários negócios/lotes ou uma fila grande de webhooks podem causar atrasos e atrasar a Meta. O Node local usa timers de cinco segundos para fila e cerca de 15 minutos para sync, com outras dimensões de lote; não reproduz automaticamente toda a política do Worker.

O runtime limita chamadas controladas a 40 por invocação, reservando margem para a plataforma. Valores comerciais de quota/preço mudam; verificar as páginas oficiais antes de dimensionar. Dry-run/local não certifica a CPU efetiva do Workers Free.

Cada leitura completa de coleção tem limite de 500 registos no adapter Worker. Acima disso retorna erro 422, em vez de truncar silenciosamente. Há paginação de integrações/catálogo, mas ainda não do estado completo do negócio. Planejar essa correção antes de aumentar o histórico de produção.

Firebase Spark já atingiu quota de leituras durante a utilização. Cache, ETag e sobreposição menor foram introduzidos para reduzir consumo, não para repor quota esgotada. Quotas contam documentos, não só chamadas HTTP. Consultar Usage e os logs antes de atribuir um erro ao login. Não autorizar upgrades por omissão.

14.2 Falhas comuns

Sintoma/código	Conferir e agir
FIREBASE_QUOTA	Usage Firestore, leitores/sync, quota e reset aplicável. Reduzir consumo; manter plano autorizado.
FIREBASE_ACCESS	Permissões IAM da conta de serviço e projeto correto; não abrir regras do cliente.
FIREBASE_IDENTITY	JSON/identidade recusada, projeto, chave e disponibilidade do serviço Google.
FIREBASE_UNAVAILABLE / TIMEOUT	Disponibilidade/rede/latência; repetir de forma controlada.
Sessão reconhecida, dados não carregam	GET /state, código de diagnóstico e limite 500; usar recuperação de sessão.
Integração precisa de administrador	Presença de ID/Secret/chave e APP_BASE_URL em produção; rede/ambiente correto.
Autorização expirada/inválida	Nova tentativa OAuth do painel; state expira/é consumido uma vez. Não recarregar callback antigo.
Domínio Shopify divergente	Domínio .myshopify.com correto e validação da identidade; não retirar HMAC.
Falha de INTEGRATION_ENCRYPTION_KEY	Base64 de 32 bytes, Secret correto, cifragem UTF-8, preservação da chave anterior.
Shopify GraphQL ACCESS_DENIED em HTTP 200	Escopos, autorização de dados protegidos e campos consultados; HTTP sozinho não prova sucesso.
Totais baixos num filtro anual	Cobertura dos registos carregados, data registada/entrega, estados/provas e statusAudit. Comparar com Shopify.
Pedido antigo indisponível	accessScopes/read_all_orders ou pedido ausente; não reinterpretar ausência como cancelado.
Concluir processa mas não paga	canMarkAsPaid/userErrors; mostrar falha e retomar apenas o estado em falta.
Campanha sem produto	Nome/alias, data final válida, conjunto cadastrado e ambiguidades.
Câmbio inesperado	Taxa guardada/conta/moeda, reimportação da janela e histórico da alteração da taxa.
409 ao guardar	Conflito de revisão ou lease; preservar dados locais e conciliar, não forçar revisão zero.
Atualização não aparece	Commit/main, deployment e painel-build, service worker/versão, API e jobs pendentes.

Logs devem mostrar códigos/etapas seguros, nunca tokens, chaves, Authorization, payloads privados completos ou clientes. Erros de provider são sanitizados. O operador deve recolher apenas a evidência mínima necessária para diagnóstico.

A última correção de contagens não valida a conta real por si. Exigir mensagem de fim/progresso, número de indisponíveis e comparação com a Shopify antes de dizer que o total de vendas do

proprietário está correto.

15. Backups, migrações e recuperação

Exportação JSON contém dados de negócio das cinco coleções. Não inclui tokens de integração nem constitui um backup administrativo integral de Firestore/Auth. Não permite clonar autenticação, Secrets, jobs, shopLinks ou a infraestrutura.

Importação normal exige conta sem dados, tamanho até dez MB, formato reconhecido e validação antes de guardar. Conserva a cópia original. Produtos legados com estoque/local são transformados em locais. Campos/IDs/datas/fotos inválidos e IDs repetidos são recusados.

Apesar da validação antecipada, a importação completa grava vários lotes: não é uma transação atómica única de todo o negócio. Uma falha de rede/conflito a meio pode exigir recuperação assistida. Não insistir numa segunda importação indiscriminada numa conta parcialmente preenchida.

A restauração dos backups da antiga limpeza só repõe registos manuais em falta, valida antes de inserir, rejeita duplicações e não substitui IDs existentes ou registos de integração. A limpeza em massa em si não existe mais.

Dados antigos de localStorage só são legíveis no mesmo domínio/navegador em que foram guardados. Usar a versão legacy para exportar se necessário; não presumir que o domínio workers.dev consegue ler outro domínio.

Recuperação de uma instalação completa exige também acesso seguro a Firebase Auth/Firestore, Secrets, apps Shopify/Meta e configuração Cloudflare. Manter uma cópia segura da chave de cifragem e plano de restauração do banco compatível com orçamento/permissões; o projeto não implementa backup automático integral de produção.

Antes de migrar ou reescrever: exportar dados, mapear IDs e aliases, confirmar datas/grupos, preservar revisions/sourceUpdatedAt ou estabelecer uma nova política explícita, inventariar ligações sem copiar tokens para ficheiros públicos e testar uma restauração de exemplo.

16. Limitações e trabalho pendente

Prioridades técnicas e funcionais:

1. Paginar /state, relatórios e consultas para remover o bloqueio de 500 documentos por coleção, sem apresentar totais parciais como completos.
2. Definir importação histórica por período, com read_all_orders e cobertura auditável. statusAudit atual revê IDs guardados; não carrega automaticamente todo o ano.
3. Concluir validação da contagem real reportada pelo proprietário após a nova verificação/autorizações. Não substituir essa validação por testes simulados.
4. Modelar pedido e linhas de forma explícita para evitar contagens mistas, comissões duplicadas e coexistência de CSV/API. Deduplicação deve preservar histórico e valores legítimos.
5. Introduzir snapshots de custos, eventos/data de pagamento/conclusão e regras de descontos/frete/impostos/reembolsos se for necessário resultado contabilístico, em vez de estimativa atual.
6. Criar movimentos de stock se for pretendida dedução automática, sempre independente da Shopify conforme a decisão de produto.

7. Melhorar justiça/prioridade do scheduler, retries/backoff, limpeza de OAuth/jobs antigos e reprocessamento administrativo de jobs falhados.
8. Versionar uma suite de navegador reprodutível, além dos scripts temporários usados nas revisões; validar dispositivos físicos e navegadores relevantes.
9. Migrar gradualmente o script clássico/handlers inline para componentes/módulos, mantendo contratos e acessibilidade; tornar CSP mais restritiva.
10. Acrescentar retenção, eliminação/exportação de conta e endpoints/políticas de conformidade exigidos pelas plataformas.
11. Se houver comercialização, implementar planos, pagamentos, limites, direitos de acesso, suporte e verificação/distribuição dos apps. Hoje não há cobrança nem bloqueio por assinatura.
12. Se necessário, suportar outras moedas Shopify, câmbio por data, múltiplas contas Ads e atribuição de campanhas; nenhuma dessas capacidades deve ser presumida pela existência de um filtro por produto.

O CSV de meses anteriores usa a data atual do registo, enquanto a coluna de data no histórico pode mostrar a origem do reagendamento. Clarificar/exportar ambas as datas é uma melhoria pendente; não usar essa diferença como prova de que o registo foi duplicado.

Outras lacunas: mais de 100 itens Shopify; fulfilment de certos terceiros; histórico maior que a janela Meta; fotos no Firestore em vez de armazenamento dedicado; reconciliação de SKU/variantes em catálogo centrado em nomes/IDs de produto; fusos e corte diário Shopify/Meta não reconciliados contabilisticamente; falta de auditoria de ações de negócio completa.

17. Como reconstruir do zero

Uma reconstrução pode trocar a stack, mas deve conservar as decisões de produto e migrar os dados com prova de equivalência. A implementação atual é uma referência, não uma obrigação de manter um HTML monolítico.

Etapa 1 — Especificação e dados

Ler este manual com o proprietário. Confirmar moeda, fuso, definição de venda, datas dos relatórios, custos, comissão por linha/pedido, stock independente, retenção e dimensão do histórico. Inventariar os registos existentes e duplicações sem apagar nada. Guardar backups privados e um plano de reversão.

Etapa 2 — Identidade e isolamento

Escolher identidade e banco compatíveis com o orçamento. Implementar UID/negócio, API autenticada, email verificado, isolamento e revisões. Testar dois utilizadores antes de importar dados reais. Manter credenciais de plataforma exclusivamente no servidor.

Etapa 3 — Domínio e funções puras

Implementar modelo pedido/linha/produto/gasto, critérios de confirmação, aliases, rateio de gastos e fórmulas. Criar os cenários da secção de testes. Definir snapshots/data de recebimento se for uma evolução autorizada; não mudar históricos silenciosamente.

Etapa 4 — Interface móvel primeiro

Construir Resumo, Pedidos e Resultados, menu de conta, pesquisa, filtros, detalhes, diálogos, meses e reagendamento. O mesmo filtro deve controlar indicadores, gráficos e lista. Incluir estados de carregamento, vazio, falha, conflito e progresso. Preservar Inter/números legíveis ou uma alteração aprovada pelo proprietário.

Etapa 5 — Shopify

OAuth/HMAC, domínio/loja canónica, token cifrado, webhooks assinados/deduplicados, importação paginada/idempotente e jobs retomáveis. Validar leitura antes de escrever. Depois implementar Concluir processado + pago, Cancelar sem refund, confirmação assíncrona e consulta de históricos autorizados.

Etapa 6 — Meta

OAuth ads_read, seleção de conta, câmbio explícito, importação por campanha/dia e IDs estáveis. Não criar vendas a partir de compras Meta. Resolver nomes/datas sem associações arbitrárias e preservar campanhas sem produto no total.

Etapa 7 — Catálogo e inventário local

Vincular IDs/títulos Shopify ao produto local, guardar aliases de renomeação e manter custo/foto/stock independente. Não transferir inventário Shopify. Se for necessário livro de stock, implementar como domínio próprio com migração explícita dos saldos locais.

Etapa 8 — Migração e comparação

Importar cópia num ambiente isolado. Comparar por pedido/linha/produto/período os estados, valores, unidades, comissão, aliases e gastos. Tratar de forma explícita duplicates CSV/API, datas reagendadas e provas antigas. Comparar contas totais e amostras com as plataformas; não apenas aparência dos gráficos.

Etapa 9 — Operação e publicação

Configurar orçamento/quotas, monitorização, retries, backups, rollback, callbacks e Secrets. Publicar uma versão de teste, validar UID/isolamento e fluxos em contas sintéticas. Fazer passagem controlada de dados e validar números reais com o proprietário antes de desativar a versão anterior.

Etapa 10 — Continuidade

Documentar versões, variáveis, contratos, migrações e diferenças em relação a este manual. Manter decisões confirmadas separadas de propostas e atualizar testes/documentação no mesmo trabalho que muda uma regra.

18. Lista de passagem ao próximo programador

O proprietário deve conceder acesso apropriado às seguintes áreas, sem enviar Secrets em chat/documentos:

- Repositório GitHub e branch de produção.
- Conta/projeto Cloudflare, deployments, bindings e logs.
- Projeto Firebase correto, Auth, Firestore, Usage, regras e IAM.
- App Shopify, distribuição, versões, permissões e loja de teste.

- App Meta, caso de uso, permissões, revisão, conta de anúncios de teste e fuso/moeda.
- Gestor/cópia segura dos Secrets, sobretudo a chave de cifragem existente.
- Backups privados de dados e critérios de retenção/recuperação.

Na primeira sessão de trabalho:

1. Confirmar branch/commit, ler schema e regras de venda e verificar documentação contra código.
2. Executar npm ci, ambiente emulado, testes e build/dry-run.
3. Conhecer a versão publicada por painel-build e verificar modo production, sem revelar Secrets.
4. Confirmar a cobertura e os estados reais dos pedidos do proprietário, especialmente o incidente anual ainda não validado.
5. Levantar dimensão das coleções/quotas antes de aumentar importação histórica.
6. Não usar pedidos de clientes para ensaios de Concluir/Cancelar nem desativar verificações para desbloquear um teste.
7. Registar qualquer mudança de regra com migração e validação do seu efeito nos históricos.

Este manual e a cópia PDF são uma fotografia do código indicado no início. A versão Markdown é a referência editável; atualizar e regenerar a cópia PDF quando houver alterações substanciais.